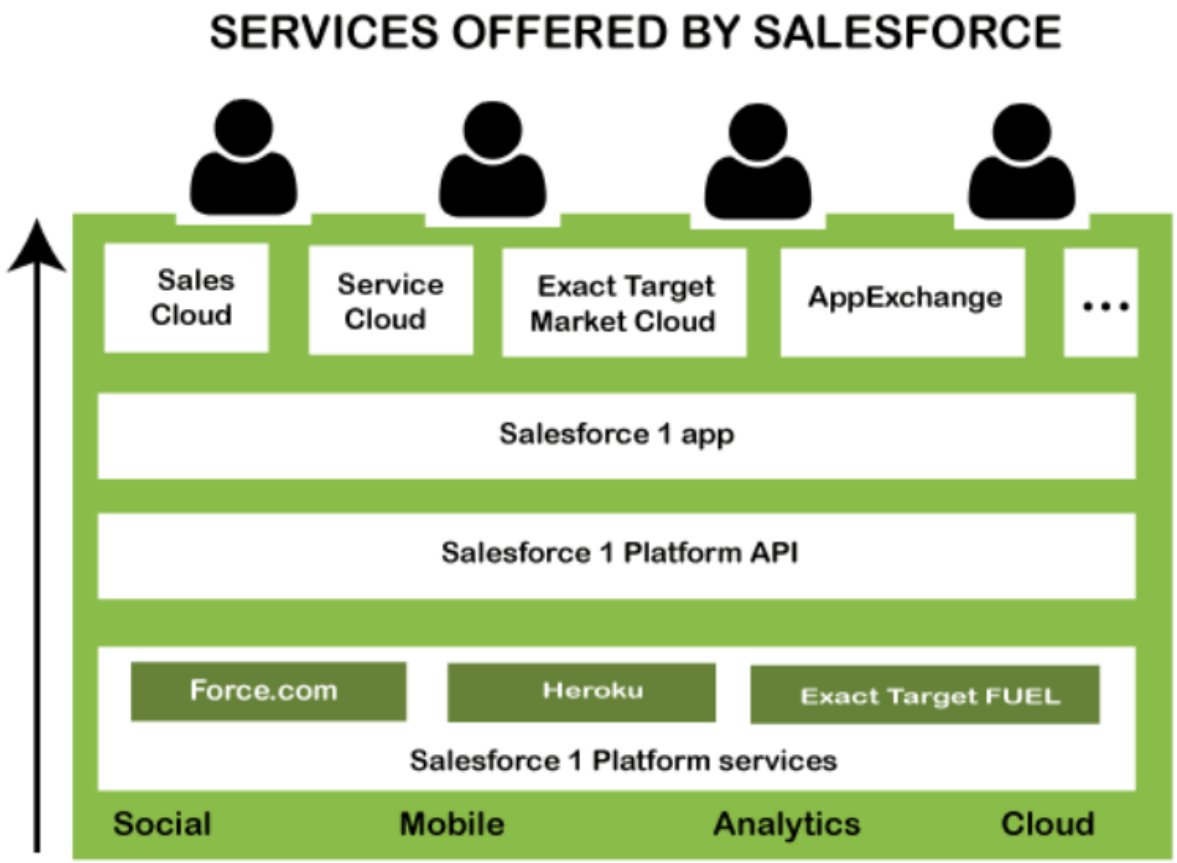
Salesforce Architecture

Salesforce is one of the leading CRM platforms to provide various customized services to its customers, partners, and employees. It also provides the platform to build custom apps, pages, components, etc., and it performs all these tasks so efficiently, mainly because of its architecture that it follows.

Salesforce Architecture is the multilayer architecture; it contains a series of layers situated on the top of each other.

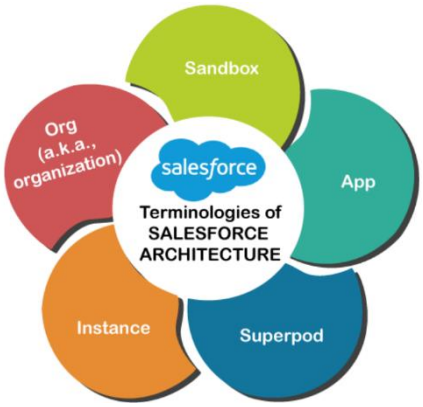
The below diagram shows the architectural view of the salesforce:



Explanation of Salesforce Architecture

- In the multilayer salesforce architecture, the users are at the topmost layer.
- The user can access a layer below the user layer, which means various clouds offered by the salesforce, such as sales cloud, service cloud, AppExchange, etc.
- The third layer is the salesforce1 App, which allows the user to access the salesforce on mobile device.
- The last layer contains various other salesforce platforms, such as Force.com, Heroku, Exact TargetFuel, etc.

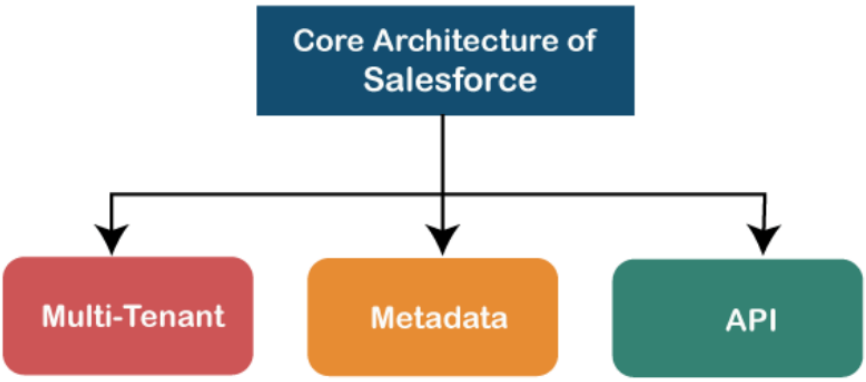
Terminologies used in Salesforce Architecture



- **App:** An app in architecture allows us to collect various things visually. The metadata elements, such as classes, objects, visual force, etc., are different from the App and independent.
- **Instance:** An instance of the salesforce architecture is the software configuration that appears in front of the user when he login to the salesforce system. It shows the server details of the particular salesforce organization on which it works. Many salesforce instances can live on a single server. However, it is based on the location of the user and changes according to user location.
- **Superpod:** Superpod is the set of frameworks and stack balancers. It includes outbound intermediary servers, system and capacity foundations, mail servers, SAN texture, and various other frameworks that support multiple instances. It provides the service isolation within a data center, so that if an issue occurs in one shared component, it may not affect every instance.
- **Org:** Org or Organization is a particular customer of a salesforce application. When a new user starts a trial on saleforce.com or developer.force.com, it generates a new org in the system. The org has customizable security and sharing settings that can be customized as per the requirement. Single org can provide support anywhere to any user whether it is multiple licensed individual user accounts, portal user accounts, or Force.com Sites user accounts.
- **Sandbox:** Sandbox is the instance of the production. It contains the sample data instead of the original data. The sandbox allows the developer to test the various conditions for the development to accomplish the client's expectations for the applications. With the sandbox, developers can create multiple copies of the production organization in different environment.

Core Architecture of Salesforce

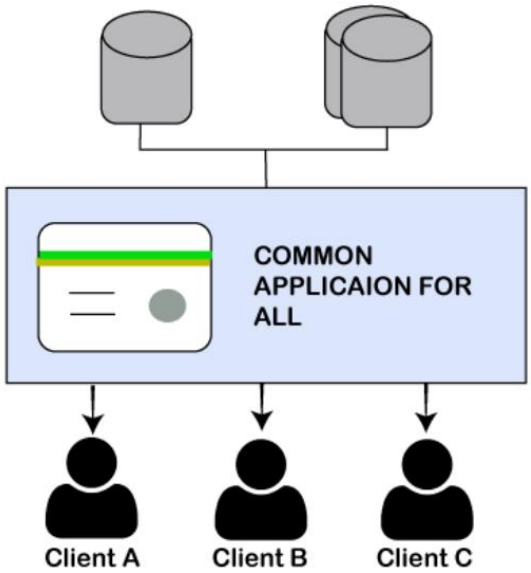
The architecture of the salesforce can be understood as a series of layers. Each layer of the architecture has different features and functionality. Each layer is described below:



1. Multi-Tenant Layer

Salesforce architecture is so popular because of its **multitenancy**. The multitenant architecture means **one common application for multiple groups or clients**. In such architecture, multiple clients use the same server, but their oaks are isolated from each other. It means the data of one client is secure and isolated from other groups or clients.

Because of multitenancy, any developer can develop an application, upload it on the cloud, and easily share it with multiple clients or groups. Multiple users share the same server and applications; hence it is very cost-effective. In salesforce, because of this multitenant architecture, all customers' data is saved in a single database.



As we can see in the above diagram, the common application is shared among the three clients.

The multitenant architecture is much efficient than single-tenant architecture. Some differences between both the architectures are given below:

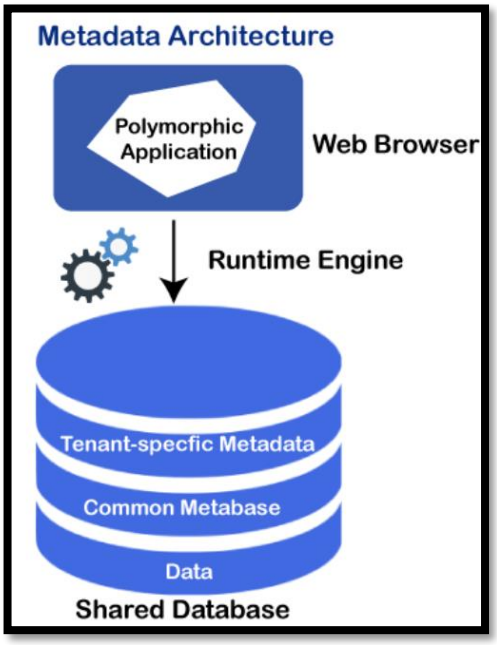
- The development cost is much high in single-tenant architecture than the multitenant because, in single-tenant, each user on the application and the maintenance cost is also owned by the single user.
- To make any update in the application, the developer needs to do it for each client manually. Whereas in multitenant, the developer needs to do it in one place, and automatically each client will receive the updated version.

2. Metadata

The Salesforce platform follows the meta-data development model. The metadata means data about the data. Salesforce stores the metadata in the shared database along with the data. It means it stores the data as well as what data does.

As we can see in the below diagram, the tenant-specific data ensures that the common data is only shared with one tenant, not with another tenant or group. This ensures the security of the data even in the shared database. The security issues get resolved with the multitenant architecture because all data is stored on different levels in the form of metadata, i.e., data above data.

We can understand it will an example, such as if there are three clients A, B, and C who contain the shared database in the salesforce platform. These groups can access their metadata from the shared data. Therefore, each client will have separate metadata. This separate metadata makes ensure each client shares his data only, not others. This increases the security of the shared database with the developer's productivity.



API Services

The salesforce metadata-driven model allows the developers to create their applications easily with the help of various tools. But sometimes developers need some more functionalities for their apps to make some modifications. To make such modifications, salesforce provides a powerful source of the APIs. These APIs helps the developers to customize the Salesforce mobile application. These APIs allows the various bits of programming to interface with each other and trade data. Without knowing many details, we can connect our apps with other apps.

The API provides a simple but powerful, and open way to programmatically access the data and any app running on the salesforce platform. These APIs helps the developers to access apps from any location, using any programming language that supports Web services, like **Java, PHP, C#, or .NET.**